

Detecting Artificial Intelligence (AI) Written Text

Jiarui Li, Ian Little, Zerui Dong

Year 4 Project
School of Mathematics
University of Edinburgh
March 6, 2025

Abstract

The universal adoption of large language models (LLMs) has made it very difficult to distinguish between human text and text generated by artificial intelligence (AI). While existing AI detection tools claim high accuracy, their methodologies are often opaque, raising concerns around their implementation. This study explores the effectiveness of machine learning-based approaches for AI text detection, with a focus on function words as features. We evaluate multiple classification models (extreme gradient boosting, support vector classifier, k-nearest neighbours, and random forest) and assess their ability to classify AI text. Through hyperparameter tuning with Bayesian optimisation, we significantly improve model performance, achieving accuracy levels exceeding 99%. Additionally, we investigate the detection of AI content when embedded within human text by generating our own AI text. To detect this embedded AI text, we adopt several sliding window approaches. Our results show that we can reach up to 97% precision when detecting different word counts of embedded AI text. This research contributes to the field of AI text detection by presenting an evaluation of machine learning classifiers when using function words as features for AI text detection.

Declaration

I declare that this thesis was composed by myself and that the work contained therein is my own, except where explicitly stated otherwise in the text.

(Jiarui Li, Ian Little, Zerui Dong)

This work is dedicated to our friends and family.

Contents

Abstract	ii
Contents	vii
1 Introduction	1
2 Background and Literature Review	3
2.1 Large Language Models (LLMs)	3
2.2 Literature Review	4
2.2.1 DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature	4
2.2.2 University of Kansas ChatGPT Detector	4
2.2.3 GLTR: Statistical Detection and Visualisation of Gener- ated Text	5
3 Mathematical Background	6
3.1 Function Words Method	6
3.1.1 Authorship Attribution	6
3.1.2 Feature Extraction	7
3.2 K-Nearest Neighbours (KNN) Classifier	9
3.2.1 Classification with KNN	9
3.2.2 Distance Metrics	9
3.2.3 Computational Efficiency	10
3.2.4 Summary	10
3.3 Random Forest (RF) Classifier	11
3.3.1 How Decision Trees Work	11
3.3.2 How Random Forests Are Trained	11
3.3.3 Why Bagging and Feature Bagging Work	12
3.3.4 Uncertainty Estimation in Random Forests	12
3.3.5 Summary	12
3.4 Extreme Gradient Boosting (XGBoost) Classifier	13
3.4.1 Regularised Learning Objective	13
3.4.2 Gradient Tree Boosting	13
3.4.3 Leaf Weight Calculation	14
3.4.4 Weighted Quantile Sketch	14
3.4.5 Controlling Overfitting in XGBoost	15

3.4.6	Summary	15
3.5	Support Vector Classifier (SVC)	16
3.5.1	Formulation	16
3.5.2	Kernel Functions	17
3.5.3	Decision Function	18
3.5.4	Summary	18
4	AI Detection Models	19
4.1	Dataset	19
4.2	Training the Models	21
4.3	Evaluation Metrics	22
4.4	Results Before Hyperparameter Tuning	22
4.5	Tuning With Bayesian Optimisation	23
4.5.1	Problem Statement	24
4.5.2	Gaussian Process (GP) as a Surrogate Model	24
4.5.3	Acquisition Function	25
4.5.4	Bayesian Optimisation Algorithm	25
4.6	Results After Hyperparameter Tuning	26
4.7	Test Set Results	27
4.7.1	Accuracy on Different Word Counts	28
4.7.2	Top Features	29
5	Detecting Embedded AI Text	31
5.1	Embedding AI Text	31
5.2	Sliding Window Detector	32
5.2.1	Formulation	33
5.2.2	Default	34
5.2.3	Majority Vote	34
5.2.4	Unanimous Vote	34
5.3	Word-level Evaluation Metrics	35
5.4	Results	36
6	Conclusions	39
6.1	Discussion	39
6.2	Limitations and Future Work	40
	Bibliography	43
	Appendices	44
A		45
A.1	Examples of Pipeline in Python Syntax	45
A.1.1	Support Vector Classifier (SVC)	45
A.1.2	K-Nearest Neighbours (KNN)	45
A.1.3	Random Forest (RF)	46
A.1.4	Extreme Gradient Boosting (XGBoost)	46
B		47

B.1	Hyperparameters Found From Tuning	47
B.1.1	Support Vector Classifier (SVC)	47
B.1.2	K-Nearest Neighbours (KNN)	47
B.1.3	Random Forest (RF)	47
B.1.4	Extreme Gradient Boosting (XGBoost)	47
C		48
C.1	Top 100 Features for XGBoost	48

Chapter 1

Introduction

The recent advancement of large language models (LLMs) has changed natural language processing, enabling artificial intelligence (AI) to generate text that is almost indistinguishable from human-written content. These models have been integrated into various domains, from content creation to customer service. Of course, LLMs offer huge potential for enhancing productivity, but their extremely popular usage has led to concerns about academic integrity and possible misinformation.

Traditional plagiarism detection tools, such as Turnitin¹, claim to achieve very high accuracy when detecting AI text [34]. However, the methodologies behind these claims remain largely undisclosed, raising concerns about their reliability and potential biases. The lack of transparency in proprietary AI detection systems indicates the need for open-source approaches to identifying AI text. In response to this challenge, our project aims to develop and evaluate machine learning-based techniques for detecting AI content.

We propose the use of the function words method, a common approach [24, 30, 3, 37] based on linguistic feature extraction, to detect AI-generated text. Using the function words approach, we aim to build a pipeline that can be used to explore various text classification models and their effectiveness in classifying AI text from human writing, examining their strengths and limitations, and testing their practical applicability in real-world scenarios.

To structure our project, this paper is outlined as follows:

Chapter 2 provides essential background and related work, outlining an overview of LLMs and exploring several published AI detection techniques.

Chapter 3 introduces the function words approach and the classification models under study, by formalising and explaining the mathematical concepts in depth.

Chapter 4 details the construction of our machine learning pipeline, covering dataset selection, preprocessing techniques, model training, hyperparameter tuning, and the final results of each classifier.

¹<https://www.turnitin.com>

Chapter 5 extends the investigation to embedded AI text detection, where we generate a dataset of mixed AI-human texts and test various sliding window methods for localising AI content within larger documents.

Through this research, we seek to contribute to the growing field of AI text detection by assessing the effectiveness of different machine learning models using the function words approach.

Chapter 2

Background and Literature Review

Since our project focuses on detecting AI-generated text, we will begin with a brief overview of large language models (LLMs), which in turn, leads to the motivation behind AI text detection. Having explained LLMs, we will then provide a literature review, whereby we will discuss some published methods of detecting AI text.

2.1 Large Language Models (LLMs)

Large language models (LLMs), such as ChatGPT¹ from OpenAI, have recently become a very popular topic due to their extraordinary abilities in processing and responding to natural language requests. They achieve the ability to generate logically coherent human language texts through extensive training on large amounts of data (billions of parameters) [36]. Today, LLMs are built upon the transformer architecture, introduced in [33], where multi-head attention layers are stacked in a very deep neural network.

In today's world, LLMs excel in text generation, as they are able to produce coherent articles, stories, and dialogues. Not only this, for machine translation, pre-trained LLMs enhance multilingual communication by accurately translating complex texts. Also, conversational AI powers virtual assistants and customer service bots everywhere.

Despite such widespread adoption, LLMs are not without limitations. They are prone to generating factually inaccurate statements (hallucinations), perpetuating societal biases present in training data, and enabling plagiarism or misinformation [21]. For instance, students might submit AI-generated essays as original work, while malicious users could automate the spread of online misinformation. These risks highlight the vital need for vigorous AI text detectors.

¹<https://chatgpt.com/>

2.2 Literature Review

Having defined LLMs and explained the motivation behind detecting AI text, we can now summarise relevant literature that explores the detection of AI-generated content. We have chosen several studies to review [23, 12, 13, 18].

2.2.1 DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature

Published in 2023, DetectGPT [23] is a popular method for identifying AI text that employs a unique property of large language models (LLMs). The study hypothesise that text produced by an LLM typically lies in regions of negative curvature in the model’s log probability function. In other words, when you make small, semantically similar alterations to AI-generated text, the log probability drops significantly compared to the original text. By contrast, human text does not exhibit such a drop.

The paper formalises this observation by approximating the trace of the Hessian (i.e. the curvature) of the log probability function using techniques such as Hutchinson’s trace estimator. DetectGPT measures the average difference in log probability between the original text and its perturbed versions. If this discrepancy exceeds a certain threshold (normalised by its standard deviation), the text is likely AI.

Extensive experiments demonstrate that DetectGPT outperforms existing detection methods, achieving significantly higher scores across a range of datasets (news articles, academic texts, creative writing) and different AI models (from 1.5B-parameter GPT-2 to 20B-parameter GPT-NeoX). Also, DetectGPT excels under various conditions, including different decoding strategies and distribution shifts, and even in black-box scenarios where the source AI model is unknown.

In summary, the paper validates the use of local log probability curvature as a discriminative signal for AI text and presents DetectGPT as a practical tool for AI text detection.

2.2.2 University of Kansas ChatGPT Detector

Researchers from the University of Kansas developed a specialised AI content detector to distinguish between human-written and AI-generated academic texts, achieving very high accuracy rates (over 99%) [12, 13].

To build the model, they trained on a modest dataset derived from 64 Perspectives articles in the journal Science (human text) paired with similar texts produced by ChatGPT, yielding 1,276 paragraphs. Next, they extracted 20 key linguistic features from each text, including sentence length variations, specific word usage, and punctuation patterns. At paragraph level the detector achieved an accuracy of 92–94%, but at document level the accuracy was 99.5% [12].

Later on, the same researchers applied the same model to a dataset specifically

tailored to chemistry papers. They first collected introduction sections from 100 published chemistry papers across 10 journals of the American Chemical Society (ACS), using them as human-written text samples. Then they used ChatGPT to generate 200 introduction sections based on the titles and abstracts of these papers, creating a dataset of AI content. The detector achieved a 100% accuracy rate in identifying AI-generated introductions based on titles and a 98% rate for those based on abstracts [13].

2.2.3 GLTR: Statistical Detection and Visualisation of Generated Text

GLTR [18] (Giant Language model Test Room) is a visual tool designed to help users detect whether given text is AI. The study argues that LLMs tend to generate text by sampling from only a small, high-probability subset of the overall distribution. This sampling bias results in distinctive features (such as unusually high token probabilities and limited lexical diversity) that can be statistically identified.

GLTR employs simple, transparent statistical tests based on token log probabilities, ranks, and entropy measures. It then overlays this information on the text with color-coded annotations, enabling users (even without prior training) to visually distinguish AI text from human content. In a human-subjects study, GLTR increased the detection accuracy of fake text from 54% to 72%.

Overall, the study demonstrates that straightforward statistical analyses, when presented visually, can be an effective means of detecting AI text, thus providing a useful approach for forensic analysis of AI text.

Chapter 3

Mathematical Background

To be able to detect AI-generated text, we need to define our feature extraction method and explain the classification models we have chosen to research. Thus, for this chapter, we will begin by explaining the function words approach used throughout the project. Then, we will detail each of the classification models we have chosen to investigate by discussing them formally and in-depth.

3.1 Function Words Method

3.1.1 Authorship Attribution

Stylometry can be defined as the statistical analysis of literary style, and literary texts [28, 22]. In this project, we will focus on the topic of *authorship attribution* which is the ability to determine who wrote a given piece of text [24]. In our case, we will be investigating the detection of AI-generated text under different conditions. To understand authorship attribution, we can define it formally.

Let $Y = \{y_1, y_2, \dots, y_{|Y|}\}$ be a set of authors. For each author $y_i \in Y$, assume we have a collection of n_i texts that are known to have been written by y_i . We denote the corpus for author y_i as,

$$c_i = (x_{i,1}, x_{i,2}, \dots, x_{i,n_i}), \quad (3.1)$$

where each text $x_{i,j}$ is paired with its label y_i . Together, the complete labelled corpus c (training set) is represented by the set of pairs,

$$\{(x_{i,j}, y_i) \mid y_i \in Y, j = 1, \dots, n_i\}. \quad (3.2)$$

Now, let $\tilde{x}$ be an anonymous text whose author is unknown. The goal is to apply a classification model (trained on the labelled corpus) such that,

$$\text{classifier}(\tilde{x}) = \hat{y} \in Y, \quad (3.3)$$

predicting the most likely author $\hat{y}$ of $\tilde{x}$. This is known as a supervised learning problem, specifically a classification problem, where the labelled corpus consists

of texts with known authorship and the task is to classify the unknown text $\tilde{x}$ among the candidate authors in Y [28]. It is important to note that for this project, our classification problem is binary: we consider authors to be either human or AI, and thus we define $Y = \{\text{human, AI}\}$ as our candidate authors.

3.1.2 Feature Extraction

To perform statistical analysis, we need to represent the text in a quantitative numerical manner. To do this, we can use feature extraction to focus on relevant features in the text. For example, we could solely focus on the frequency of rare words. However, this does not work well in practice as it is impossible to define a set of rare words applicable to every piece of text. The function word approach, widely used in stylometry [24, 30, 3, 37], focuses only on the frequency with which common words occur, based on the idea that individuals exhibit subtle variations in their use of these function words. Within the classic study into authorship of the disputed federalist papers [24] we can see that 70 function words (seen in figure 3.1) were used.

a, all, also, an, and, any, are, as, at, be, been, but, by, can, do, down, even, every, for, from, had, has, have, her, his, if, in, into, is, it, its, may, more, must, my, no, not, now, of, on, one, only, or, our, shall, should, so, some, such, than, that, the, their, then, there, things, this, to, up, upon, was, were, what, when, which, who, will, with, would, your

Figure 3.1: List of 70 Function words used in [24].

With this defined set of common words, we can transform any text into a vector of 70 components, where each component is the frequency of a corresponding function word. For example, the text (seen in figure 3.2) taken from the previously mentioned federalist paper study [24] would be extracted, using the 70 function words (seen in figure 3.1) as features, as the feature vector displayed in figure 3.3.

As statisticians, our interest in this controversy is largely methodological. Problems of discrimination are widespread, and we wished a case study that would give us an opportunity to compare the more usual methods of discrimination with an approach via Bayes' theorem.

Figure 3.2: Example text extract taken from [24].

Alternatively, we could also use the most commonly used words from a given corpus c and use this as our feature extraction. However, this can cause issues with overfitting¹ and subsequently a classification model could become ‘corpus-specific’.

¹Classification model is too specific on dataset so doesn't generalise well to unseen data.

0, 0, 0, 2, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1,
 0, 0, 0, 1, 0, 0, 0, 0, 0, 2, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1,
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0

Figure 3.3: Vector representation of example text (seen in figure 3.2) when using the 70 function words (seen in figure 3.1) from [24].

Furthermore, we know that each text in a given corpus will exhibit different word counts. Therefore, we must normalise each vector x'_i . This new vector can be calculated as follows,

$$x_i = \frac{x'_i}{\sum_{j=1}^D x'_{i,j}}, \quad (3.4)$$

where D is the number of features (defined function words). The example vector seen in figure 3.3 would therefore become the vector displayed in figure 3.4.

0, 0, 0, 0.12, 0.06, 0, 0.06, 0.06, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0.06, 0, 0.06, 0, 0, 0, 0.06, 0, 0, 0, 0, 0, 0.12, 0, 0, 0, 0, 0.06, 0, 0, 0, 0,
 0, 0, 0.06, 0.06, 0, 0, 0, 0, 0.06, 0.06, 0, 0, 0, 0, 0, 0, 0, 0.06, 0.06, 0

Figure 3.4: Normalisation of feature vector (seen in figure 3.3) using equation (3.4) rounded to two decimal places.

Once we have normalised all vectors we can train a classification model on the vectors of both AI and human authors to allow us to predict the author of any given text. Instead of using the 70 function words (seen in figure 3.1), for this project we will use the 100 most common words from English language Twitter [28] which are displayed in figure 3.5.

the, i, to, and, a, of, in, that, it, my, is, you, for, was, on, me, but, so,
 this, with, have, be, we, at, not, all, he, im, as, like, are, just, its, out, up,
 about, they, what, or, one, if, do, from, had, get, when, will, there, her,
 dont, she, time, know, now, can, some, then, by, his, really, no, well, an,
 your, go, more, were, am, think, would, who, people, good, been, how,
 has, got, him, them, going, because, back, day, see, much, our, only,
 which, want, their, love, even, other, too, after, today, went, over, way,
 here

Figure 3.5: The 100 most common words from English language Twitter [28].

3.2 K-Nearest Neighbours (KNN) Classifier

First of all, we decided to use k-nearest neighbours (KNN) [17] as it is commonly used in the task of authorship attribution, an example being [2]. KNN is an extremely simple supervised learning method used for classification. It classifies new observations by considering the k closest labelled examples in the training set. The choice of k is a hyperparameter that plays an important role in model performance: a smaller k may make the model highly sensitive to noise, leading to overfitting, while a larger k smooths decision boundaries but may cause underfitting.

3.2.1 Classification with KNN

Let N denote the number of training vectors such that $x_1, \dots, x_N$, with corresponding class labels (human or AI) $y_1, \dots, y_N$. To classify a new observation $\tilde{x}$, we calculate the distance d between $\tilde{x}$ and every observation x_i in the training set. The distance function, denoted as $d(x_i, \tilde{x})$, measures the similarity between each training point and $\tilde{x}$, and the closest neighbours are identified based on this measure. Mathematically,

$$d_i = d(x_i, \tilde{x}), \quad (3.5)$$

where d is a distance function. The choices for d include the $L1$ norm (Manhattan distance), $L2$ norm (Euclidean distance), Lp norm (Minkowski distance), and Cosine distance. In the simplest case, when $k = 1$, the predicted class $\hat{y}$ for a new observation $\tilde{x}$ is assigned based on the class label y_r of the nearest neighbour,

$$r = \arg \min_i d_i, \quad (3.6)$$

where r denotes the index of the closest training example, thus $\hat{y}$ corresponds to the label of the r -th observation y_r in the training examples. However, for $k > 1$, the classifier identifies the k nearest neighbours and assigns the new observation to the class based on the labels of these k neighbours. To determine this class, there is a hyperparameter which dictates the weighting scheme. The most simple weighting scheme is called uniform weighting, where all neighbours contribute equally (majority voting). For example, if $k = 5$ and three out of five neighbours belong to a certain class, the new observation $\tilde{x}$ is therefore assigned to that class. Alternatively, distance-based weighting assigns greater importance to closer neighbours. Thus, distance-weighted voting can be particularly useful in scenarios where the density of data points varies in the feature space, helping reduce misclassification from distant but highly influential points.

3.2.2 Distance Metrics

The choice of distance metric, used to determine d_i , is also a hyperparameter that significantly impacts classification accuracy. Euclidean distance is commonly used when features have similar importance and scale, while Manhattan distance may be preferable in high-dimensional sparse data. More generally, the Minkowski

distance allows flexibility by adjusting the exponent p , effectively interpolating between these two metrics. There is also Cosine distance, which is often used when the orientation of the vectors is more important than their magnitude. Formally, each of the distance functions can be defined as follows, let D denote the total number of features then for Manhattan distance ($L1$),

$$d_{L1}(x_i, \tilde{x}) = \sum_{j=1}^D |x_{i,j} - \tilde{x}_j|. \quad (3.7)$$

The Euclidean distance ($L2$) is denoted,

$$d_{L2}(x_i, \tilde{x}) = \sqrt{\sum_{j=1}^D (x_{i,j} - \tilde{x}_j)^2}. \quad (3.8)$$

Next, the Minkowski distance (Lp generalised form),

$$d_{Lp}(x_i, \tilde{x}) = \left(\sum_{j=1}^D |x_{i,j} - \tilde{x}_j|^p \right)^{\frac{1}{p}}, \quad (3.9)$$

note that the Minkowski distance is described as a generalisation of both Manhattan and Euclidean distances because for $p = 1$, it simplifies to the Manhattan distance, and when $p = 2$, it becomes the Euclidean distance. Lastly, the Cosine distance is,

$$d_{cos}(x_i, \tilde{x}) = 1 - \frac{\sum_{j=1}^D x_{i,j} \tilde{x}_j}{\left(\sqrt{\sum_{j=1}^D x_{i,j}^2} \right) \left(\sqrt{\sum_{j=1}^D \tilde{x}_j^2} \right)}, \quad (3.10)$$

measuring the dissimilarity between two vectors based on the cosine of the angle between them.

3.2.3 Computational Efficiency

Computational efficiency is another important aspect of KNN, particularly for large datasets. A brute-force approach, which calculates distances from all training samples, is feasible for a small dataset but becomes more difficult as the dataset grows. There exist more efficient alternatives, such as KD-tree and Ball-tree, that help accelerate neighbour searches by organising data in a structured way. These methods significantly reduce the number of comparisons needed when retrieving the nearest neighbours, improving the classification speed.

3.2.4 Summary

The KNN classifier is a simple but effective method. Its performance depends heavily on hyperparameters, which control the number of neighbours, distance metric, weighting scheme, and search algorithm [35, 20, 17, 29, 10].

3.3 Random Forest (RF) Classifier

The Random Forest Classifier is a machine learning algorithm that uses an ensemble of multiple decision trees to classify data [5]. It has been explored in terms of authorship attribution, for example [27]. These models operate by asking a sequence of yes/no questions about the input data, ultimately leading to a prediction. This structure mirrors human decision-making, where conclusions are drawn step by step through logical inquiries.

3.3.1 How Decision Trees Work

The process of constructing a decision tree involves splitting the data based on questions that maximise predictive accuracy. The CART (Classification and Regression Tree) algorithm is often used, selecting splits that minimise Gini impurity, a measure of how mixed the classes are within a node. The aim is to group similar data points together, creating “pure” subsets.

The tree recursively splits the dataset based on selected features until one of the following conditions is met:

- All data points in a node belong to the same class.
- The tree reaches a maximum depth defined by the `max_depth` hyperparameter.
- A minimum number of data points, described by the `min_samples_split` hyperparameter, per node is reached.

To move from decision trees to random forests, random forests build upon decision trees by employing the ensemble learning technique of bootstrap aggregating (bagging). This approach enhances performance by combining predictions from multiple trees, reducing variance while keeping bias low.

3.3.2 How Random Forests Are Trained

Bootstrap Sampling

Given a training dataset $X = \{x_1, x_2, \dots, x_N\}$ with corresponding labels $Y = \{y_1, y_2, \dots, y_N\}$, the training process of a random forest is as follows:

1. For each tree $b = 1, \dots, B$ (where B is the total number of trees):
 - A new training dataset X_b, Y_b is created by randomly sampling n training examples with replacement.
 - A decision tree f_b is trained on X_b, Y_b .

After training, predictions for an unseen observation $\tilde{x}$ are aggregated:

- **For regression:** The final prediction is the average of all tree predictions,

$$\hat{f}(\tilde{x}) = \frac{1}{B} \sum_{b=1}^B f_b(\tilde{x}). \quad (3.11)$$

- **For classification:** The final output is determined by majority voting.

Feature Bagging

Random forests introduce an additional layer of randomness by selecting a random subset of features at each split, which reduces tree correlation and improves performance. Let D denote the total number of features.

- For classification problems, $\lfloor \sqrt{D} \rfloor$ features are typically selected at each split.
- For regression, $\lfloor D/3 \rfloor$ features are used, with a recommended minimum node size of 5.

3.3.3 Why Bagging and Feature Bagging Work

- **Bagging:** Decorrelates trees by training each one on a different bootstrap sample. Without this step, training on the same dataset would result in highly correlated trees, reducing the effectiveness of the ensemble.
- **Feature Bagging:** Prevents over-reliance on dominant features, promoting tree diversity and further reducing correlations.

Together, these techniques decrease model variance and improve generalisation, making random forests more robust compared to single decision trees.

3.3.4 Uncertainty Estimation in Random Forests

Random forests can also estimate prediction uncertainty. The standard deviation of the predictions from all trees provides a confidence measure,

$$\sigma = \sqrt{\frac{1}{B-1} \sum_{b=1}^B \left(f_b(\tilde{x}) - \hat{f}(\tilde{x}) \right)^2}. \quad (3.12)$$

3.3.5 Summary

The Random Forest classifier uses bagging and feature bagging to build an ensemble of decorrelated decision trees, reducing variance while maintaining low bias. Key hyperparameters include the number of trees B , the number of features sampled per split (e.g. $\lfloor \sqrt{D} \rfloor$ for classification), and tree depth constraints (e.g. maximum depth or minimum samples per leaf). By aggregating predictions across trees, random forests achieve robustness against overfitting and provide uncertainty estimates through prediction variance [5].

3.4 Extreme Gradient Boosting (XGBoost) Classifier

The Extreme Gradient Boosting (XGBoost) model is an efficient tree-boosting algorithm commonly used for classification tasks. While it has been less frequently explored in authorship attribution, its widespread success in various machine learning applications [8] makes it a compelling candidate for investigation in this context. Compared to traditional boosting methods, it offers high speed and accuracy, particularly for large-scale datasets. One of its key strengths is the ability to handle missing values and sparse data efficiently, making it highly adaptable to real-world scenarios.

3.4.1 Regularised Learning Objective

To prevent overfitting, XGBoost optimises a regularised objective function, which is defined as,

$$L(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k), \quad (3.13)$$

where y_i represents the true label of the i -th sample, $\hat{y}_i$ denotes the model's predicted value for the i -th sample, l is a differentiable convex loss function that measures the difference between the prediction and the actual value and, $\Omega(f_k) = \gamma T + \frac{1}{2}\lambda\|w\|^2$ is a regularisation term to control model complexity.

Within equation (3.13), XGBoost controls model complexity using a regularisation term $\Omega(f)$, which helps to prevent overfitting. The regularisation term is defined as,

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_j w_j^2, \quad (3.14)$$

where T is the number of leaves in the tree, controlled by the penalty parameter γ , w_j represents the weight of leaf j , and λ controls L2 regularisation. The inclusion of γ ensures that only splits that significantly reduce the loss function are performed, thereby controlling tree growth and reducing model complexity. The L2 regularisation parameter λ helps in smoothing the weights of the leaves, making the model more robust to noise in the training data.

3.4.2 Gradient Tree Boosting

Similar to random forest (RF) (defined in section 3.3), XGBoost also employs a tree-based boosting method. It builds trees step by step, with each new tree improving on the mistakes of the previous ones. At each step t , the model minimises,

$$L^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t), \quad (3.15)$$

where f_t is the new function learned at step t . Using a second-order approximation, the objective simplifies to,

$$L^{(t)} \approx \sum_{i=1}^n \left[l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t), \quad (3.16)$$

where g_i and h_i are the first and second derivatives of the loss function. Thus, each new tree f_t leverages both gradient information (direction) and Hessian information (step size) to improve model convergence.

3.4.3 Leaf Weight Calculation

XGBoost determines the optimal leaf node weights by solving for the best weight w_j that minimises the objective function. The optimal weight w_j^* is given by,

$$w_j^* = - \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}, \quad (3.17)$$

where I_j is the set of training samples falling into leaf j , the numerator sums the first-order gradients g_i over all samples in the leaf and the denominator sums the second-order derivatives h_i , plus the regularisation term λ .

The gain from splitting a node is computed as,

$$Gain = \frac{1}{2} \left[\frac{\left(\sum_{i \in I_L} g_i \right)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{\left(\sum_{i \in I_R} g_i \right)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{\left(\sum_{i \in I} g_i \right)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma, \quad (3.18)$$

where I_L and I_R are the left and right child nodes, the first two terms compute the gains for the left and right nodes, the third term represents the original node's contribution before splitting and, γ is the penalty for adding a new leaf. A split occurs only if the gain is positive, ensuring that the model benefits from the split.

3.4.4 Weighted Quantile Sketch

XGBoost implements a weighted quantile sketch to efficiently determine split points in large datasets. It defines a rank function r_k for each feature k ,

$$r_k(z) = \frac{1}{\sum_{(x,h) \in D_k} h} \sum_{(x,h) \in D_k, x < z} h, \quad (3.19)$$

where D_k represents the training instances for feature k with second-order gradient statistics h . The algorithm ensures consecutive split points satisfy,

$$|r_k(s_{k,j}) - r_k(s_{k,j+1})| < \epsilon, \quad (3.20)$$

maintaining a balance between speed and accuracy.

3.4.5 Controlling Overfitting in XGBoost

XGBoost applies multiple techniques to prevent overfitting and enhance generalisation. These techniques are controlled by hyperparameters that can be defined as follows:

Shrinkage (Learning Rate η)

Instead of applying full updates, XGBoost scales each tree's contribution by a learning rate η ,

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta f_t(x_i). \quad (3.21)$$

A smaller η leads to slower but more stable training, requiring more trees.

Subsampling

A fraction of the training data is randomly selected at each boosting iteration (e.g., `subsample = 0.8` means 80% of data is used per tree). This prevents the model from relying too heavily on specific patterns, thereby reducing overfitting.

Column Sampling

Instead of using all features, XGBoost selects a random subset of features for each tree (e.g., `colsample_bytree = 0.7` means 70% of features are used per tree). This reduces feature correlation and improves generalisation.

L1 and L2 Regularisation

- **L1 Regularisation (α):** Encourages sparsity by setting some weights to zero.
- **L2 Regularisation (λ):** Smooths weight values, preventing excessive complexity.

These mathematical principles make XGBoost an efficient, stable, and well-suited machine learning algorithm for large-scale data.

3.4.6 Summary

XGBoost is a gradient-boosting framework that iteratively builds decision trees to correct errors from previous iterations. Its regularised objective function (equation (3.13)) combines loss minimisation with penalties on tree complexity (γ), leaf weights (λ), and model updates (learning rate η). Advanced features like weighted quantile sketches, column/row subsampling, and support for missing data make the classifier efficient [8].

3.5 Support Vector Classifier (SVC)

The support vector classifier (SVC) is an instance of the support vector machine (SVM) paradigm commonly used for the task of authorship attribution, an example being [14]. An SVM constructs a hyperplane in a high-dimensional feature space that separates data points into different classes (in our case, AI or human text). The optimal hyperplane is chosen to maximise the margin, i.e., the distance between the hyperplane and the nearest training data points of any class. A larger margin generally corresponds to a lower generalisation error. Since our problem is not linearly separable, we employ a soft-margin approach where certain training points (the support vectors) are allowed to violate the margin constraints via slack variables.

3.5.1 Formulation

Formally, given N training vectors $x_1, \dots, x_N$, with two labels (human or AI), these corresponding labels $y_1, \dots, y_N$ are converted such that for any y we have $y \in \{-1, +1\}$. We wish to find parameters w and b such that the decision function,

$$f(x) = w^T \phi(x) + b, \quad (3.22)$$

correctly classifies most training samples, where $\phi(x)$ maps x into a (potentially higher-dimensional) feature space.

To allow for non-separable data, we introduce a slack variable $\xi_i \geq 0$ for each training sample x_i . The slack variable ξ_i measures the degree to which the margin constraint is violated. Specifically,

- If $y_i f(x_i) \geq 1$, then $\xi_i = 0$;
- If $y_i f(x_i) < 1$, then $\xi_i > 0$, and if $\xi_i > 1$, the sample is misclassified.

The primal optimisation problem is then,

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i, \quad (3.23)$$

subject to the constraints,

$$y_i (w^T \phi(x_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, N. \quad (3.24)$$

Here, $C > 0$ is a hyperparameter that controls the trade-off between maximising the margin and minimising the slack penalty.

The problem in equation (3.23) has a dual representation, which is particularly useful when using kernel functions. The dual problem is given by,

$$\min_{\alpha} \frac{1}{2} \alpha^T Q \alpha - e^T \alpha, \quad (3.25)$$

subject to,

$$\sum_{i=1}^N \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C, \quad i = 1, \dots, N. \quad (3.26)$$

In this formulation, $\alpha = [\alpha_1, \dots, \alpha_N]^T$ are the dual coefficients, e is a vector of ones, and the matrix Q is defined by,

$$Q_{ij} = y_i y_j K(x_i, x_j), \quad (3.27)$$

with the kernel function $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$.

3.5.2 Kernel Functions

The choice of a kernel function $K(x_i, x_j)$ is an important hyperparameter for support vector machines. The choices include the linear, polynomial, radial basis function (RBF), and sigmoid kernels. In essence, a kernel function allows the SVC to operate in a (potentially) higher-dimensional feature space without computing the mapping $\phi(\cdot)$ explicitly. To understand the difference between each of these kernel functions, we can define them formally.

The linear kernel is often used when the data are already believed to be linearly separable (or close to it). Therefore,

$$K(x_i, x_j) = x_i^\top x_j, \quad (3.28)$$

which corresponds to a (potentially high-dimensional) dot product between the two feature vectors.

Next, the polynomial kernel introduces non-linearity by raising the dot product to a power d . A common form is,

$$K(x_i, x_j) = (x_i^\top x_j + c)^d, \quad (3.29)$$

where d is the degree of the polynomial (an integer ≥ 1) and $c \geq 0$ is a constant added to the dot product to allow for more flexible decision boundaries by adjusting the influence of higher-degree terms.

Moreover, sometimes referred to as the Gaussian kernel, the radial basis function (RBF) kernel can be denoted as,

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2), \quad (3.30)$$

where $\gamma > 0$ controls the width of the kernel. Larger values of γ make the decision boundary tighter around individual data points, whereas smaller values of γ produce smoother, broader decision boundaries.

Finally, the sigmoid kernel, stemming from neural network theory, has the form,

$$K(x_i, x_j) = \tanh(\alpha x_i^\top x_j + r), \quad (3.31)$$

where α and r are kernel parameters. Although less common in practice than RBF or polynomial, it can sometimes perform well.

3.5.3 Decision Function

After solving the dual problem in equation (3.25), the decision function for a new observation $\tilde{x}$ is given by,

$$f(\tilde{x}) = \sum_{i \in SV} \alpha_i y_i K(x_i, \tilde{x}) + b, \quad (3.32)$$

where the summation is taken over the set SV of support vectors (i.e. those training samples with $\alpha_i > 0$).

The predicted class $\hat{y}$ for $\tilde{x}$ is therefore determined by the sign of the decision function from equation (3.32),

$$\hat{y}(\tilde{x}) = \text{sign}(f(\tilde{x})). \quad (3.33)$$

3.5.4 Summary

To summarise, the support vector classifier (SVC) works by mapping training data into a high-dimensional feature space via $\phi(\cdot)$, finding a hyperplane that maximises the margin while allowing some misclassifications through slack variables, and using the dual formulation with a kernel function to efficiently compute the decision boundary. The final classification of new samples is made based on the sign of the decision function in equation (3.33) [9, 4, 31].

Chapter 4

AI Detection Models

In this chapter, we will discuss the process of transforming the previously defined classification models into effective AI text detection models. We will begin by examining the dataset used for training and evaluation, followed by a discussion on constructing a pipeline for model training. Next, we will discuss our framework for producing optimal hyperparameters and finally, we will analyse the results of each classification model (from chapter 3) on the test set.

4.1 Dataset

In order to evaluate the function words feature extraction on each of the previously defined classification models, we need a large dataset containing human and AI texts. After some research, we discovered the dataset, titled *AI Vs Human Text* [19] on Kaggle¹, a platform comprised of free-to-use datasets. Upon inspection, it contained 487,235 text entries, amassing a size of 1.11GB. Each essay was stored in a *‘text’* field and was tagged either 1 (indicating AI) or 0 (indicating human) in a *‘generated’* field. Inspecting the dataset allowed us to see that 62.76% of data entries were human text (305,797), subsequently 37.24% were AI texts (181,438).

Initially, we identified several essay entries that contained little to no text. To maintain a meaningful dataset, we removed all essays with fewer than 50 words. Next, to create a balanced dataset, we extracted an equal proportion of AI and human essays. Specifically, we included all AI essays (181,353 after removing short essays) and randomly selected 181,353 unique human texts, resulting in a dataset of 362,706 essays with a perfect 50%-50% split between AI and human text. Figure 4.1 plots the word count distributions of AI essays (orange) and human essays (blue) in the balanced dataset. Overall, the AI text distribution is more sharply peaked in the 300-400 word range, whereas the human texts exhibit a slightly broader spread, with a peak around 200-450 words and a noticeably heavier tail extending beyond 900 words. In other words, most AI texts cluster around a moderate length, while the human essays exhibit higher word counts.

¹<https://www.kaggle.com>

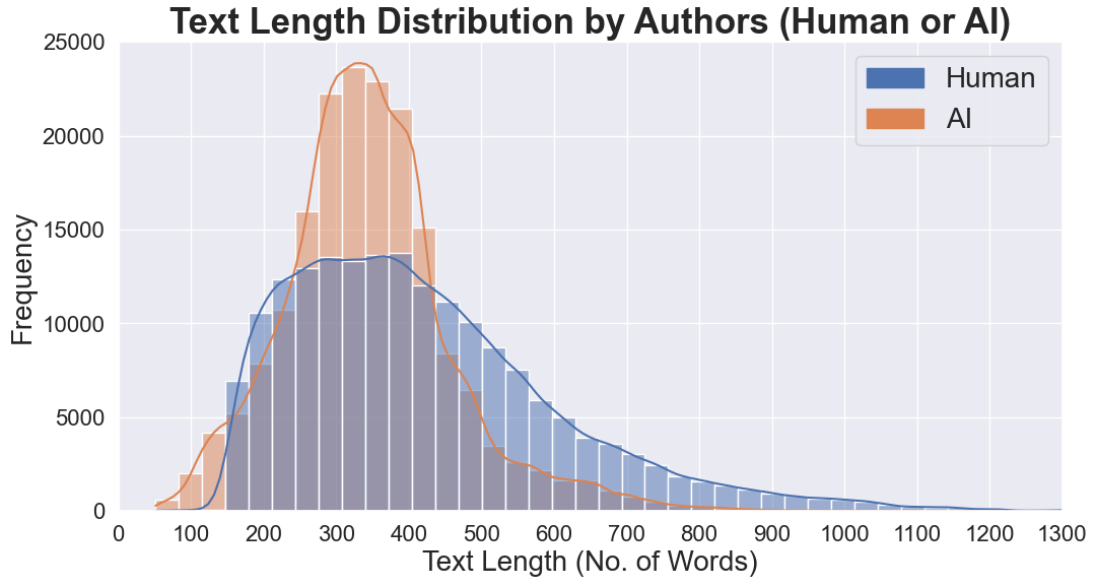


Figure 4.1: The word count distributions of the balanced dataset by author.

Furthermore, to illustrate the differences between human and AI texts, we used a multidimensional scaling (MDS) plot to compress the high dimensional relationships into 2 dimensions. We did this by randomly sampling 5,000 texts from our balanced dataset and then vectorising each with the 100 function words (seen in figure 3.5) feature extraction outlined in section 3.1. We then applied MDS to the resulting feature vectors to produce the scatter plot shown in figure 4.2, where both axes (*Dimension 1 and 2*) are arbitrary.

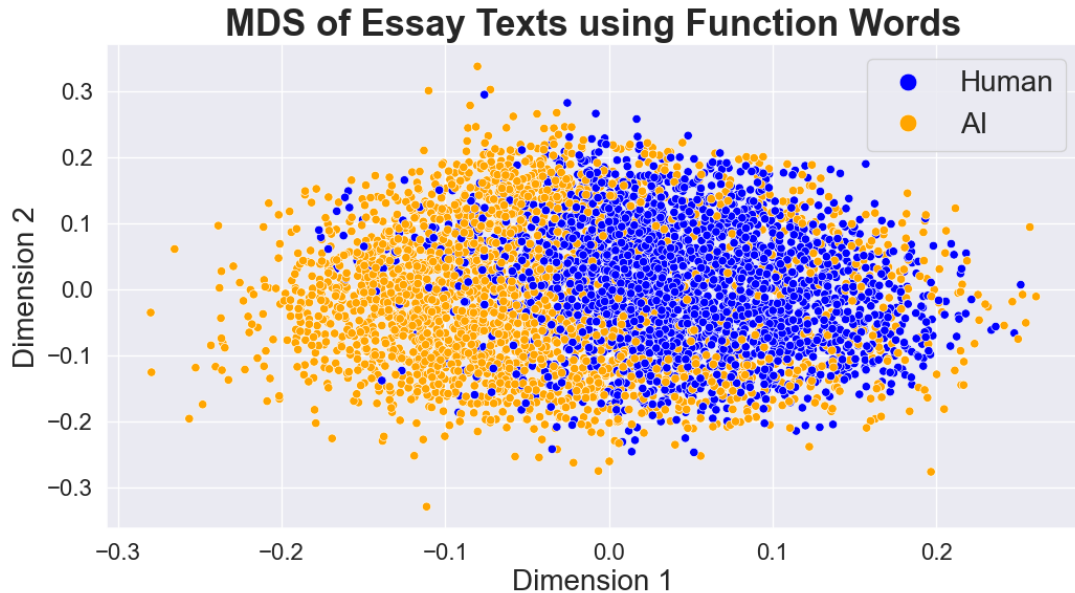


Figure 4.2: MDS plot of balanced dataset text when using function words feature extraction.

It is important to note that MDS reduces the 100 dimension relationships into 2 dimensions, which means that some of the original distance relationships inevitably get distorted. Nevertheless, figure 4.2 tells us that although there is no complete separation between AI and human texts, we do see areas dominated by one colour. In particular, the majority of the left-hand side is orange (AI) and the right-hand side is human (blue) which suggests that the function words approach does achieve some separation.

4.2 Training the Models

We first split the balanced dataset randomly into a **training-set** (80% - 290,164 essays), **validation-set** and a **test-set** (each 10% - 36,271 essays). Next, we were able to implement the function words method described in section 3.1 by training each classification model (using the training set) through the pipeline in figure 4.3. The steps highlighted in green denote the preprocessing (function words feature extraction) and the step in blue represents the deployment of a classification model on the normalised feature vectors.

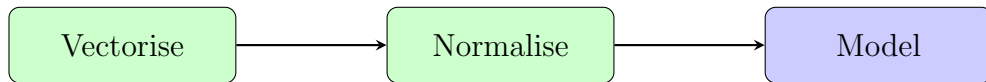


Figure 4.3: Visualisation of function words text classification pipeline.

From section 3.1 we know that the first step of the pipeline is to vectorise the text using the 100 most common words from English language twitter (see figure 3.5). To do this, we used `CountVectorizer`² from scikit-learn [25, 6] which was able to vectorise the text (using the words from figure 3.5) effectively by automatically removing special characters, punctuation, and emojis while converting all text to lowercase. Incorporating this into our pipeline eliminated the need to separately clean the dataset which ensured consistent and accurate tokenisation on all text (even unseen data). Moreover, considering section 3.1 again we know that the next step of the pipeline is to normalise the feature vectors we have just extracted from the text. To implement this, we used another scikit-learn [25, 6] function called `Normalizer`³ to standardise feature vectors in the same way as equation (3.4). Lastly, the final stage of the pipeline is to train a classification model on these normalised feature vectors. All models discussed in this paper (defined in chapter 3) follow this procedure; an example of each classification model's pipeline in Python syntax can be seen in appendix A.1.

At first, we attempted to train each model on our local machines, but due to the size of the dataset, it was taking an exceedingly long time. In example, the SVC model was taking over a day to train. To tackle the massive wait time we applied for and gained access to the university's graphics processing unit (GPU) cluster EDDIE [15]. With GPU access, we could now employ the same models

²[CountVectorizer documentation.](#)

³[Normalizer documentation.](#)

but with GPU accelerated training. For XGBoost⁴ we need only activate the GPU predictor within the model, but for all other classifiers (SVC, KNN and RF) we needed to find GPU accelerated alternatives since scikit-learn [25, 6] does not boast GPU support. Eventually, we discovered the cuML library [26] by NVIDIA, which is comprised of GPU accelerated models that are capable of much faster training than the same models we had run locally. For instance, the KNN model from cuML has shown to be up to 600 times faster than its CPU-based counterpart [11]. After setting up the cuML library, we were able to train each one much quicker than before; SVC in particular was reduced from over a day to train to now training in a matter of minutes.

4.3 Evaluation Metrics

Before we report our results, we need to define the evaluation metrics we will consider when evaluating our models' performance. The metrics we consider include precision, recall, F1-score, and accuracy, with support indicating the number of instances in each class.

Let, TP be the number of true positives (e.g. AI text correctly classified as AI), FP be the number of false positives (e.g. human text incorrectly classified as AI), TN be the number of true negatives (e.g. human text correctly classified as human), and FN be the number of false negatives (e.g. AI text incorrectly classified as human). Using these definitions, the evaluation metrics are computed as follows,

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (4.1)$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad (4.2)$$

$$\text{F1-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (4.3)$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (4.4)$$

In other words, precision is the proportion of instances classified as belonging to a given class that are actually correct. Recall is the proportion of actual instances of a class that the model correctly identifies. F1-Score is the harmonic mean of precision and recall which provides a single metric that balances both the correctness of the predictions (precision) and the completeness (recall). Lastly, accuracy is the overall proportion of correct predictions made by the model.

4.4 Results Before Hyperparameter Tuning

First of all, we trained the models (using the training set - 290,164 essays) without specifying any hyperparameters and so each model used its default settings. The

⁴[XGBoost documentation](#).

results of this experiment, on the validation set (36,271 essays), can be seen in table 4.1.

Model	Class	Precision	Recall	F1-Score	Support
XGBoost	Human	0.9782	0.9796	0.9789	18212
	AI	0.9794	0.9780	0.9787	18059
	Accuracy			0.9788	36271
SVC	Human	0.9630	0.9646	0.9638	18212
	AI	0.9643	0.9626	0.9634	18059
	Accuracy			0.9636	36271
KNN	Human	0.9727	0.9852	0.9789	18212
	AI	0.9849	0.9721	0.9785	18059
	Accuracy			0.9787	36271
RF	Human	0.9818	0.9846	0.9832	18212
	AI	0.9844	0.9816	0.9830	18059
	Accuracy			0.9831	36271

Table 4.1: Classification reports for each model using default hyperparameters on validation set (36,271 essays).

Across all models, we observe high classification accuracies of over 96%. The random forest (RF) classifier achieves the pinnacle accuracy at 98.31%, followed by XGBoost at 97.88% and KNN at 97.87%. The SVC model performs slightly worse, with an accuracy of 96.36%, though still competitive. Interestingly, the precision-recall balance remains consistent across models, indicating that no model significantly favours one class over the other.

The differences in model performance may be attributed to how each algorithm processes features. Random forest and XGBoost, both ensemble-based models, likely benefit from their ability to capture complex decision boundaries. KNN also performs well, suggesting that document representations in the chosen feature space exhibit strong locality properties. SVC, despite being theoretically effective for high-dimensional text classification, may be underperforming due to the non-linearly separable nature of the data. Overall, these results establish a strong baseline, but further improvements may be possible through hyperparameter tuning, which we explore in the next section.

4.5 Tuning With Bayesian Optimisation

As discussed throughout chapter 3, each model has various hyperparameters that can be adjusted to enhance performance. The process of finding the best combination of these hyperparameters is known as *hyperparameter tuning*. To avoid finding hyperparameters that overfit on the test set, it is crucial to tune hyperparameters using the validation set before applying them to the unseen test set.

This approach helps prevent overfitting and ensures that the selected hyperparameters generalise well to new data. To perform hyperparameter tuning, we invoked Bayesian optimisation through the framework Optuna [1].

4.5.1 Problem Statement

Optimising an objective function can be a challenging task, especially when the function is computationally expensive and lacks an explicit analytical form. Bayesian optimisation (BO) offers an efficient solution by employing probabilistic modelling to reduce the number of expensive function evaluations while effectively searching for the optimal solution [32]. Based on Bayesian inference, Bayesian optimisation typically uses Gaussian processes (GP) as a surrogate model to approximate the unknown objective function. By utilising an Acquisition Function to iteratively select the most informative points, Bayesian optimisation balances exploration (searching new promising regions) and exploitation (refining the current best solution), allowing it to find the optimum with fewer function evaluations. The goal of Bayesian optimisation is to optimise an unknown objective function $f(x)$, which is defined over a constrained search space $\mathcal{X}$,

$$x^* = \arg \max_{x \in \mathcal{X}} f(x), \quad (4.5)$$

where x represents the input parameters, $f(x)$ is the computationally expensive objective function (training the classification model) and $\mathcal{X}$ is the constrained search space (hyperparameter values). Since evaluating $f(x)$ is costly, Bayesian optimisation constructs a probabilistic model $\hat{f}(x)$ to approximate it, enabling a more efficient search for the optimum.

4.5.2 Gaussian Process (GP) as a Surrogate Model

A Gaussian process (GP) is a distribution over functions, defined by a mean function $m(x)$ and a covariance function (kernel) $k(x, x')$,

$$f(x) \sim \mathcal{GP}(m(x), k(x, x')), \quad (4.6)$$

where $m(x)$ is the mean function, often set to 0 for simplicity, and $k(x, x')$ is the covariance function, which determines the smoothness of the function. Given observed data $\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n$, where $y_i = f(x_i) + \epsilon$ with noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$, the posterior distribution of the objective function is given by,

$$p(f(x) \mid \mathcal{D}_n) = \mathcal{N}(\mu_n(x), \sigma_n^2(x)), \quad (4.7)$$

where the mean function (expected value) is,

$$\mu_n(x) = k(x, X_n)K_n^{-1}Y_n. \quad (4.8)$$

The variance function (uncertainty estimation) is,

$$\sigma_n^2(x) = k(x, x) - k(x, X_n)K_n^{-1}k(X_n, x), \quad (4.9)$$

where X_n represents the set of evaluated points, K_n is the covariance matrix $[k(x_i, x_j)]_{i,j=1}^n$, and Y_n is the vector of observed values $[y_1, \dots, y_n]$. This posterior distribution provides both an estimate of the objective function at any point x and a measure of uncertainty.

4.5.3 Acquisition Function

A critical aspect of Bayesian optimisation is the selection of the next point x_{n+1} for evaluation, which is determined by an *acquisition function* $a(x)$. Several common acquisition functions include the following.

Expected Improvement (EI) selects the point that maximises the expected improvement over the current best observed value f^+ ,

$$EI(x) = \mathbb{E}[\max(f(x) - f^+, 0)]. \quad (4.10)$$

Under the Gaussian process posterior, this expectation can be expressed as,

$$EI(x) = (\mu_n(x) - f^+) \Phi(Z) + \sigma_n(x) \phi(Z), \quad (4.11)$$

where $Z = \frac{\mu_n(x) - f^+}{\sigma_n(x)}$, $\Phi(Z)$ is the cumulative distribution function (CDF) of the standard normal distribution and $\phi(Z)$ is the probability density function (PDF) of the standard normal distribution.

Probability of Improvement (PI) selects the point with the highest probability of improving upon f^+ ,

$$PI(x) = P(f(x) > f^+) = \Phi\left(\frac{\mu_n(x) - f^+}{\sigma_n(x)}\right). \quad (4.12)$$

The Upper Confidence Bound (UCB) approach selects the point that maximizes a weighted sum of the mean estimate and an exploration term,

$$UCB(x) = \mu_n(x) + \kappa \cdot \sigma_n(x), \quad (4.13)$$

where κ is a tunable parameter that balances exploration and exploitation.

4.5.4 Bayesian Optimisation Algorithm

Bayesian optimisation follows an iterative procedure,

Initialisation: Select an initial set of sample points $\mathcal{D}_0$ for evaluation.

Fit Gaussian Process: Train the Gaussian process model using the available data $\mathcal{D}_n$.

Select Next Point: Determine the next evaluation point by maximising the acquisition function $a(x)$,

$$x_{n+1} = \arg \max_{x \in \mathcal{X}} a(x). \quad (4.14)$$

Evaluate Objective Function: Compute $f(x_{n+1})$ and update the dataset,

$$\mathcal{D}_{n+1} = \mathcal{D}_n \cup \{(x_{n+1}, f(x_{n+1}))\}. \quad (4.15)$$

Repeat steps until a stopping criterion is met, such as reaching a computational budget or convergence.

Bayesian optimisation is a powerful technique for optimising expensive black-box functions. By leveraging Gaussian processes to model the function and acquisition functions to balance exploration and exploitation, it efficiently finds the optimal solution with fewer function evaluations [32].

4.6 Results After Hyperparameter Tuning

To improve model performance, we applied Bayesian optimisation, performing 500 iterations for KNN and XGBoost, 200 for Random Forest, and 147 for SVC. The difference in iteration counts can be attributed to the differences in computational cost; we initially aimed for 500 iterations per model but imposed a 24-hour runtime limit. The optimal hyperparameters found for each model are listed in appendix B.1. Table 4.2 provides the results of each model, on the validation set, after tuning.

Model	Class	Precision	Recall	F1-Score	Support
XGBoost	Human	0.9959	0.9953	0.9956	18212
	AI	0.9953	0.9959	0.9956	18059
	Accuracy			0.9956	36271
SVC	Human	0.9918	0.9912	0.9915	18212
	AI	0.9911	0.9917	0.9914	18059
	Accuracy			0.9914	36271
KNN	Human	0.9990	0.9967	0.9978	18212
	AI	0.9967	0.9989	0.9978	18059
	Accuracy			0.9978	36271
RF	Human	0.9911	0.9934	0.9922	18212
	AI	0.9933	0.9910	0.9922	18059
	Accuracy			0.9922	36271

Table 4.2: Classification reports for each model on the validation set (36,271 essays) when using hyperparameters found from tuning.

When compared to the baseline performance from table 4.1, all models show improvements across all evaluation metrics (accuracy, precision, recall, and F1-score). In terms of accuracy, all models now achieve over 99% when classifying the validation set with the highest performing model being KNN which boasts an incredible accuracy of 99.78%.

We now refer to table 4.3 which summarises the accuracy improvements before and after hyperparameter tuning. SVC benefits the most from tuning as its accuracy increases by 2.78%, followed by KNN (1.91%) and XGBoost (1.68%). Random Forest, which already had strong performance, saw a smaller but meaningful increase of 0.91%. Additionally, the fact that RF, despite its strong default performance, showed the least improvement suggests that its default hyperparameters were already appropriate for this task and so it was not very sensitive to hyperparameter tuning. Conversely, the large improvement in SVC’s accuracy implies that its default settings were not ideal, emphasising the importance of hyperparameter tuning for this model.

Model	Default Accuracy (%)	Tuned Accuracy (%)	Improvement (%)
XGBoost	97.88	99.56	+ 1.68
SVC	96.36	99.14	+ 2.78
KNN	97.87	99.78	+ 1.91
RF	98.31	99.22	+ 0.91

Table 4.3: Improvement of each classification models’ accuracy before and after hyperparameter tuning.

The performance improvement across all models shows the effectuality of Bayesian optimisation in finding well-suited hyperparameters. As the gains were consistent across precision, recall, and F1-score (not just accuracy) it implies the hyperparameters may find a better generalisation rather than overfitting.

4.7 Test Set Results

After tuning hyperparameters on the validation set (see appendix B.1 for found hyperparameters), we deployed said hyperparameters on the test set to evaluate performance on unseen data. Table 4.4 summarises the classification metrics for each model.

The results indicate only a slight decrease in accuracy from the validation to the test set across all models. This minor drop suggests that the chosen hyperparameters generalise well, and the models are robust against overfitting. Considering all four models, the accuracy scores achieved by both KNN and XGBoost decrease the least (0.01% and 0.03% respectively) when comparing the validation and test set results.

Moreover, the similar performance levels across different classifiers imply that the selected features (100 function words) are strong predictors regardless of the model type. While there are slight differences (KNN and XGBoost marginally outperforming SVC and RF) these variations are minimal, indicating that each tuned model effectively captures the underlying distinctions between human and AI text. The successful application of hyperparameter tuning (using Optuna) further demonstrates the utility of automated optimisation in enhancing model performance.

Model	Class	Precision	Recall	F1-Score	Support
XGBoost	Human	0.9953	0.9952	0.9953	18187
	AI	0.9952	0.9953	0.9952	18084
	Accuracy	0.9953			36271
SVC	Human	0.9894	0.9912	0.9903	18212
	AI	0.9911	0.9893	0.9902	18059
	Accuracy	0.9902			36271
KNN	Human	0.9988	0.9966	0.9977	18187
	AI	0.9966	0.9988	0.9977	18084
	Accuracy	0.9977			36271
RF	Human	0.9893	0.9923	0.9908	18187
	AI	0.9922	0.9892	0.9907	18084
	Accuracy	0.9908			36271

Table 4.4: Classification reports for each model on the test set (36,271 essays) when using hyperparameters found from tuning.

4.7.1 Accuracy on Different Word Counts

To assess how each classifier performs across different text lengths in the test set, we analyse the bar chart in figure 4.4.

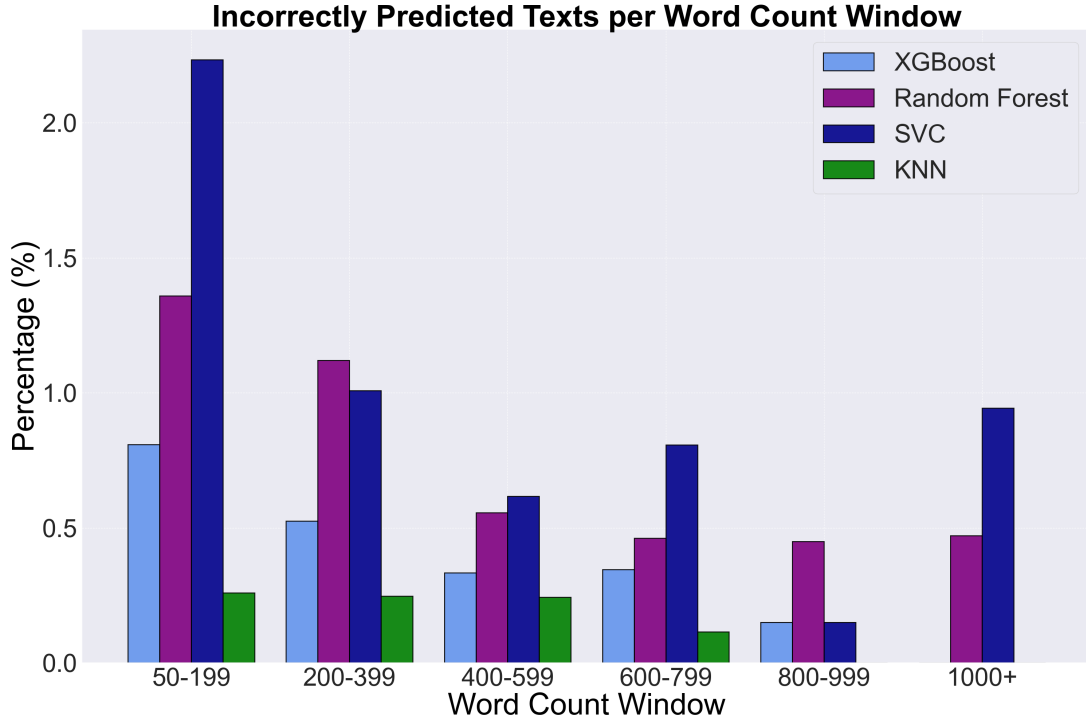


Figure 4.4: The percentage of texts, for different word count windows, that were incorrectly predicted by each classification model on the test set.

As expected, all models exhibit higher misclassification rates on shorter texts. However, an interesting exception is KNN, which maintains strong performance across all text lengths. Given that KNN achieved the highest overall accuracy on the test set, this consistency is not entirely surprising. It is also worth mentioning that both XGBoost and KNN correctly classify all texts over 1,000 words, suggesting that both models substantially benefit from more context.

Both random forest (RF) and the support vector classifier (SVC) initially show a clear trend of improved classification as text length increases. However, this improvement reverses when handling texts exceeding 1,000 words, where misclassification rates rise again. This suggests that while longer texts generally provide more context for classification, excessively long texts may introduce challenges such as overfitting or increased complexity, particularly for SVC.

4.7.2 Top Features

Understanding which features contribute most to AI text detection is essential for refining feature extraction in classification models. Unlike SVC and KNN, both XGBoost and Random Forest provide intrinsic measures of feature importance, allowing us to analyse which function words play a significant role in distinguishing AI-generated text from human writing. For our analysis, we extracted the top 10 most influential function words using the XGBoost model. Figure 4.5 presents the feature importance scores, ranking each function word based on its contribution to the model's predictive performance on the test set.

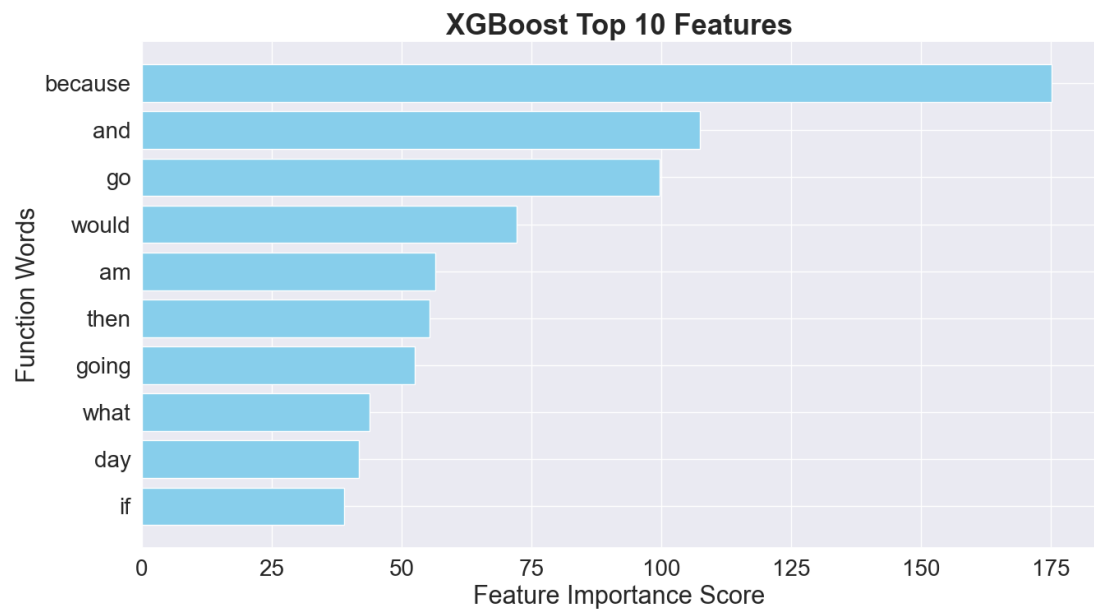


Figure 4.5: XGBoost top 10 most important function words (features) when predicting the test set.

Notably, the word *because* emerges as the most critical indicator, with a significantly higher importance score compared to all other function words. This substantial difference indicates that the model relies heavily on *because* to distinguish between human and AI text. Similarly, words like *and* and *go* rank significantly high among the top contributors, highlighting more function words that exhibit major differences when used by AI models. Interestingly, certain function words, such as *i* and *a* (see appendix C.1 for all words), were assigned an importance score of zero, meaning that they provide no useful information for classification. This could be attributed to their high frequency and uniform distribution in both AI and human-generated text, making them ineffective as statistical features. Using these insights, future work can refine the number of function words actually needed for AI detection. A refinement which would reduce the computational cost of some of the investigated models.

Chapter 5

Detecting Embedded AI Text

Our previous AI detection models classified entire texts as either human or AI. Of course, when it comes to real-life plagiarism detection, AI content often appears interwoven with human writing. Since our earlier models were trained on uniformly labelled texts, they cannot reliably detect segments of AI text embedded within human text. In this chapter, we introduce several *sliding window* approaches to identify AI-generated portions within mixed texts. We begin by describing the process we used to generate our own AI text. Next, we outline the experimental setup for creating mixed (AI and human) text samples. Finally, we detail and evaluate each sliding window method for detecting embedded AI content.

5.1 Embedding AI Text

To produce a dataset of essays with embedded AI text, we used OpenAI’s API¹ to generate AI text. When generating text, we leveraged the system prompt which tells the large language model (LLM) how it should behave, providing good control over its output. We designed the system prompt (visible in figure 5.1) to ensure the LLM was capable of producing a specified number of additional words.

You are a system that, when given a piece of text, you continue that text for x number of words. Only return the continued text.

Figure 5.1: OpenAI system prompt for generating continued text when given human test set samples.

To use this system prompt (seen in figure 5.1), we randomly extracted 1,000 human essays (average word count of 425.4 words) from the test set and gave each to OpenAI’s gpt-4o-mini² model, as it was the most suitable for our use

¹<https://openai.com/index/openai-api/>

²<https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>

case. The model was instructed to continue each essay by $x = \{200, 500, 1000\}$ words, making sure to store all the AI continued text. Upon inspecting the dataset of AI generated text, we noticed that when asked to generate x number words, the model struggled to adhere to a strict word count. For 200 words, the model generated, on average, text with a word count of 228.6 words; for 500 words, the average output was 569.7 words, and for 1,000 words, the model output an average of 878.3 words. Nevertheless, the word counts demonstrate a consistent increase as $x \in \{200, 500, 1000\}$ increases, which is the primary concern for the experiment.

Now that we have 200, 500, and 1,000 word AI continuations for each of the 1,000 sampled human essays, we can outline the experiment designed to assess our methods for detecting embedded AI text. Quite simply, we created test samples by appending each AI-generated continuation to its corresponding human-written essay, resulting in three extended texts per human-written essay. To illustrate the test samples, we can refer to figure 5.2 which showcases the change point³ in one of the test samples.

...it's best to have a good impression because it can help you through life with people you met before. Good impression have a lot of good benefits with your social life. So the next time when you meet someone new It's good to have a good impression. In addition, making a positive first impression can have long-lasting effects on personal and professional relationships. For instance, in the workplace, your initial interactions with colleagues and supervisors can shape their perception of your work ethic...

Figure 5.2: Example extract taken from test sample where human text (blue) is continued by AI (red).

These modified texts serve as input for the sliding window detection methods, which will be introduced in the next section. Since we have the gold standard human and AI text stored separately, we can leverage word-level evaluation metrics to assess the accuracy and effectiveness of each sliding window approach in identifying embedded AI text.

5.2 Sliding Window Detector

To detect AI text within mixed (human and AI) documents, we employ a sliding window detector that moves through the text, classifying segments individually to localise AI content. Figure 5.3 from [7] visualises the sliding window approach by representing each word in a piece of text as $W1, \dots, W24$. The sliding window method is then defined by a window size and a step size, which dictate its behaviour when moving through text.

³A point in which the text changes from human to AI (or vice versa).

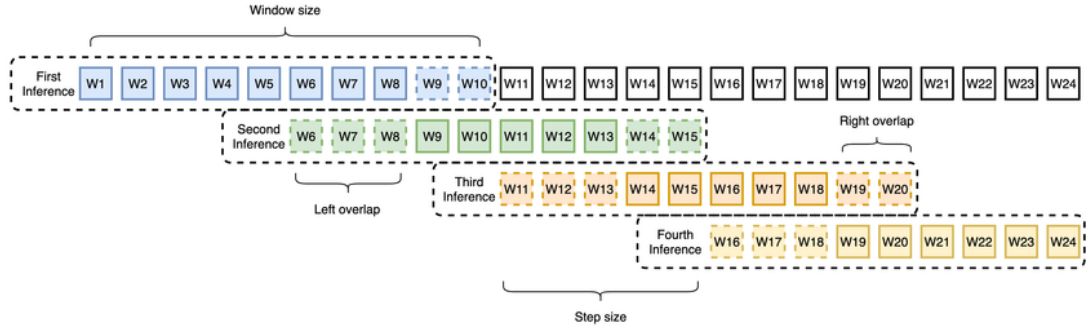


Figure 5.3: Visualisation of sliding window (from [7]) moving through text and classifying windows.

5.2.1 Formulation

To fully understand the sliding window, we can define it formally. Let,

$$T = [t_1, t_2, \dots, t_n], \quad (5.1)$$

be a piece of text consisting of n words. The sliding window technique is described by both the window size (w), the fixed number of consecutive words included in each window, and the step size (s), the number of words by which the window is shifted at each step. For $k = 1, 2, \dots, m_0$, where,

$$m_0 = \left\lfloor \frac{n - w}{s} \right\rfloor + 1, \quad (5.2)$$

we can define the starting index of the k -th window as,

$$i_k = 1 + (k - 1)s. \quad (5.3)$$

Thus, the k -th window is given by,

$$W_k = [t_{i_k}, t_{i_k+1}, \dots, t_{i_k+w-1}], \quad (5.4)$$

for all k such that $i_k + w - 1 \leq n$. In cases where $i_{m_0} + w - 1 < n$, there remain words at the end of the text that are not covered by the sliding windows. To ensure full coverage of T , we include an extra window defined as,

$$W_{m_0+1} = [t_{n-w+1}, t_{n-w+2}, \dots, t_n]. \quad (5.5)$$

Thus, the total number of windows, m , is,

$$m = \begin{cases} m_0 + 1, & \text{if } i_{m_0} + w - 1 < n, \\ m_0, & \text{otherwise.} \end{cases} \quad (5.6)$$

For each window W_k ($k = 1, 2, \dots, m$), our chosen model predicts the text as 0 (human) or 1 (AI). These window-level classifications form the groundwork for our sliding window detection approaches, which include the default, majority vote, and unanimous vote.

5.2.2 Default

The default sliding window approach is the most basic method for detecting AI-generated text within a document. As described previously, the text is split into m windows, each of fixed size w . For each window W_k ($k = 1, 2, \dots, m$), a classifier is used to determine whether the window is AI or human. In the default approach, every window that is classified as AI is retained without any further steps. That is, the final set of detected AI windows is given by,

$$\{W_k \mid \text{classifier}(W_k) = 1\}. \quad (5.7)$$

Thus, the default method provides a straightforward way to localise AI content by directly aggregating all windows that the model deems as AI-generated.

5.2.3 Majority Vote

The majority vote sliding window method extends the default approach by aggregating predictions at the word level, rather than at the window level. Instead of simply returning every window classified as AI, this method evaluates each word based on the multiple windows in which it appears. For each word t_i in the text $T = [t_1, t_2, \dots, t_n]$, let,

$$\mathcal{K}_i = \{k \mid t_i \in W_k\}, \quad (5.8)$$

denote the set of windows that include t_i . The classifier predicts each window W_k as 0 (human) or 1 (AI). The word-level prediction $g(t_i)$ is then determined by a majority vote over the windows containing t_i ,

$$g(t_i) = \begin{cases} 1, & \text{if } \sum_{k \in \mathcal{K}_i} \text{classifier}(W_k) > \frac{|\mathcal{K}_i|}{2}, \\ 0, & \text{otherwise.} \end{cases} \quad (5.9)$$

To put it another way, a word t_i is classified as AI only if more than half of the windows in which it appears are predicted as AI. This aggregation helps to mitigate errors that might occur from misclassification in individual windows, leading to a more robust identification of AI text.

5.2.4 Unanimous Vote

Considering a more conservative approach, the unanimous vote sliding window method is an alternative to the majority vote approach. Like before, we first partition the text $T = [t_1, t_2, \dots, t_n]$ into m windows of fixed size w . For each window W_k ($k = 1, 2, \dots, m$), a classifier predicts the window W_k as 0 (human)

or 1 (AI). For each word t_i in the text, just as in the majority vote, let,

$$\mathcal{K}_i = \{k \mid t_i \in W_k\}, \quad (5.10)$$

denote the set of windows that include t_i . In the unanimous vote approach, a word is classified as AI-generated only if *all* windows in which it appears are predicted as AI. Formally, the word-level prediction $h(t_i)$ is defined as,

$$h(t_i) = \begin{cases} 1, & \text{if } \sum_{k \in \mathcal{K}_i} \text{classifier}(W_k) = |\mathcal{K}_i|, \\ 0, & \text{otherwise.} \end{cases} \quad (5.11)$$

In other words, if even a single window that contains t_i is classified as human, then t_i is not considered to be AI. Thus, this version of sliding window is particularly stringent and can help reduce false positives by ensuring that only words consistently identified as AI across all contexts are flagged as such.

5.3 Word-level Evaluation Metrics

Similar to the previous chapter, we need to define our evaluation metrics in order to assess the performance of our sliding window detectors. At a finer granularity, we evaluate the sliding window detectors at the word level. In this context, each word is treated as an individual instance, and its label is determined based on the aggregation of sliding window predictions. We compare these word-level predictions against the ground truth labels.

Let, TP be the number of true positives (words that are truly AI and identified as AI), FP be the number of false positives (words that are human but detected as AI), TN be the number of true negatives (words that are human and classified as human), and FN be the number of false negatives (words that are AI but predicted as human). Using these definitions, the word-level evaluation metrics are computed, just as before,

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (5.12)$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad (5.13)$$

$$\text{F1-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (5.14)$$

$$\text{F1-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (5.15)$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (5.16)$$

Essentially, precision is now the proportion of words predicted as AI that are indeed AI. It measures the reliability of the AI label among the words flagged by the detector. Recall becomes the proportion of actual AI words that the sliding window approach successfully identifies. It reflects the detector's ability

to capture all AI-generated words. The F1-Score is still the harmonic mean of precision and recall. It provides a balanced measure of the detector’s performance, especially when there is an imbalance between the number of AI and human words. Finally, accuracy is now the overall proportion of words that are correctly classified by the sliding window approach.

5.4 Results

To investigate each of the previously defined sliding window approaches, we deployed each of them on the test samples (human essays with 200, 500, or 1,000 AI embedded words). We chose a sliding window with a window size of 250 words and a step size of 50 words to ensure consistency across all experiments. Of course, in an ideal world, we would run the sliding window with a step size of 1, but even with access to the GPU cluster, this was not computationally possible. It is also important to note that, due to a compute unified device architecture (CUDA) memory error, we could not get the random forest (RF) classifier to work with the unanimous vote sliding window.

We first evaluated the default sliding window approach on each test sample. The results for this experiment are presented in table 5.1, where the values in bold highlight the highest scores for that evaluation metric and AI word count.

Model	AI Word Count	Accuracy	Precision	Recall	F1 Score
XGBoost	200	0.755	0.611	0.892	0.715
SVC	200	0.775	0.612	0.842	0.701
KNN	200	0.809	0.659	0.920	0.762
RF	200	0.731	0.594	0.896	0.702
XGBoost	500	0.829	0.782	0.916	0.838
SVC	500	0.843	0.801	0.896	0.840
KNN	500	0.870	0.824	0.947	0.878
RF	500	0.814	0.767	0.916	0.830
XGBoost	1000	0.848	0.820	0.908	0.858
SVC	1000	0.860	0.836	0.896	0.861
KNN	1000	0.891	0.863	0.946	0.900
RF	1000	0.839	0.808	0.914	0.854

Table 5.1: Combined classification report for default sliding window on 200, 500 and 1,000 words of embedded AI text.

As expected, increasing the AI word count improves both accuracy and precision for all classification models. This suggests that when the document contains a higher proportion of AI text, the classifiers can more effectively distinguish AI content from human text. When considering the best performing classifier, the answer is very clearly KNN which achieves the highest score across every single evaluation metric. A result which correlates directly with the test set evaluation in section 4.7 which resulted in KNN as top performer.

However, a major drawback of the default approach is its relatively low precision, particularly for the 200-word samples. Thus, there’s a high false positive rate, meaning that a substantial portion of human-written text is incorrectly classified as AI-generated. Such behaviour is problematic in real-world applications where falsely flagging human content can undermine AI detection. Given these limitations, we explore more conservative strategies such as the majority vote and unanimous vote approaches in an attempt to reduce false positives.

We next evaluated the majority vote sliding window approach, which considers the number of times a word appears in AI-classified windows by choosing only words that appear in a majority of AI windows. The results are summarised in table 5.2.

Model	AI Word Count	Accuracy	Precision	Recall	F1 Score
XGBoost	200	0.809	0.786	0.694	0.683
SVC	200	0.803	0.792	0.606	0.631
KNN	200	0.860	0.861	0.724	0.745
RF	200	0.789	0.755	0.704	0.672
XGBoost	500	0.872	0.867	0.854	0.853
SVC	500	0.877	0.905	0.833	0.857
KNN	500	0.924	0.947	0.904	0.918
RF	500	0.859	0.845	0.856	0.842
XGBoost	1000	0.881	0.885	0.870	0.872
SVC	1000	0.884	0.916	0.853	0.874
KNN	1000	0.927	0.951	0.911	0.924
RF	1000	0.864	0.857	0.861	0.853

Table 5.2: Combined classification report for majority vote sliding window on 200, 500 and 1,000 words of embedded AI text.

Just like in the default approach results (seen in table 5.1), KNN again emerges as the highest performer across every single metric and AI word count. In contrast to the default approach, the majority vote method aggregates window-based classifications at the word level, leading to an overall improvement in precision and accuracy across every classifier. For instance, at 200 words, KNN’s precision exhibits a significant increase in precision across all word counts, with a gain of over 20 percentage points at 200 words (from 65.9% to 86.1%), making it the top performer in this setting. Similarly, SVC and XGBoost show steady improvements in precision at higher word counts, with SVC reaching 91.6% precision at 1,000 words.

However, this gain in precision comes at the cost of recall reduction. For example, XGBoost’s recall drops from 89.2% in the default method to 69.4% when using majority vote at 200 words. This trend persists across all models and word counts, indicating that while majority vote reduces false positives, it does so by increasing false negatives, which makes the classifiers less sensitive to detecting every instance of AI text.

Finally, we evaluate the most conservative approach, unanimous vote sliding window which only flags words as AI if they are classified as AI in every window. The results of which are shown in table 5.3.

Model	AI Word Count	Accuracy	Precision	Recall	F1 Score
XGBoost	200	0.772	0.860	0.489	0.579
SVC	200	0.763	0.854	0.413	0.517
KNN	200	0.791	0.915	0.473	0.595
XGBoost	500	0.819	0.911	0.728	0.797
SVC	500	0.798	0.935	0.673	0.765
KNN	500	0.821	0.966	0.708	0.796
XGBoost	1000	0.833	0.911	0.776	0.829
SVC	1000	0.802	0.940	0.717	0.793
KNN	1000	0.808	0.973	0.721	0.797

Table 5.3: Combined classification report for unanimous vote sliding window on 200, 500 and 1,000 words of embedded AI text.

As expected, the unanimous vote approach results in a substantial increase in precision across all models (compared to majority vote results in table 5.2). This trend is most evident in the KNN classifier, where precision reaches 97.3% for 1,000-word samples, the highest among all tested methods. Not to mention, KNN also achieves over 91% precision across all word counts. However, the unanimous vote approach shows the first instances of the KNN classifier being outperformed on an evaluation metric, where XGBoost emerges as the best model for recall across all AI word counts. For 1,000 words of embedded AI, we also see that XGBoost achieves the highest accuracy and F1-score, but it gets the lowest precision out of all models at 91.1%.

Moreover, this further gain in precision, again, comes at a significant cost - both accuracy and recall decline compared to the majority vote method (see table 5.2). The decrease in recall is particularly severe at lower AI word counts; for instance, recall for SVC drops from 60.6% (majority vote) to 41.3% (unanimous vote) at 200 words. This suggests that, just like the majority vote approach, the unanimous vote approach further minimises false positives, by further increasing false negatives, meaning a considerable amount of embedded AI text remains undetected.

The unanimous vote method is thus best suited for scenarios where avoiding false accusations is more critical than detecting every instance of AI content. This makes it ideal for settings like plagiarism detection in academia, where mistakenly flagging human content could have extremely serious consequences.

Chapter 6

Conclusions

6.1 Discussion

In this study, we investigated various machine learning techniques for detecting AI-generated text, focusing on function words as key features. Our findings demonstrate that classification models, particularly XGBoost and KNN, achieve high accuracy in the detection of AI text, with significant performance improvements observed through Bayesian optimisation. The test set results suggest that the function words method is exceptional at distinguishing between human and AI text regardless of the chosen classification model.

Additionally, we explored the detection of embedded AI text within human writing by generating our own AI text. Using different sliding window techniques, our experiments showed that the majority vote approach provides a balanced and effective method for identifying AI-generated content within mixed documents, while the unanimous vote approach maximises precision at the cost of lower recall. These findings indicate that the choice of detection strategy should depend on the application context. We conclude that situations requiring high precision (e.g. plagiarism detection) may benefit from a unanimous vote approach, while applications demanding higher recall (e.g. content moderation) may find the default or majority vote method more suitable.

Overall, we observed that KNN, after hyperparameter tuning, was the best classification model across all scenarios. Not only did KNN achieve the highest scores on the test set, it also consistently achieved the best results across almost all evaluation metrics (most importantly, precision) throughout each of the investigated sliding window approaches.

6.2 Limitations and Future Work

While our classification models demonstrated strong performance, several limitations warrant discussion.

Firstly, although our models successfully detected our own AI-generated text (using gpt-4o-mini), they were trained on generic AI text rather than on data specific to particular AI models (e.g. gpt-4o). Future work should evaluate the performance of these classifiers across a broader range of AI models (such as DeepSeek¹) to determine whether a single unified classifier can generalise effectively or if separate models are required for the detection of different AI models.

Moreover, the high computational demands of training certain models, particularly the support vector classifier (SVC), pose another limitation. As noted in section 4.2, we had to employ GPU-accelerated methods to avoid prohibitively long training times on local machines. This dependency on expensive GPU-supported hardware could limit the feasibility of real-time detection in practical applications, especially when scaling to larger datasets.

Thirdly, our hyperparameter tuning process was constrained by a pre-set time limit. As mentioned in section 4.6, both the SVC and Random Forest (RF) models did not complete the full 500 iterations of Bayesian optimisation due to a 24-hour runtime constraint. Since these models were the lowest performers on the test set, it remains an open question whether allowing them more iterations could have improved their accuracy to levels comparable with even the KNN classifier. Future research should investigate whether extending the tuning process might enhance the performance of these models.

Next, in chapter 5 we evaluated the sliding window experiment by generating an AI continuation for human essays in the test set and appending it to the corresponding human essays. Although this method effectively assesses each sliding window approach, it does not fully represent real-world scenarios where AI text is interwoven with human text (i.e. with human writing both preceding and following the AI segment). In real situations, AI text is likely to appear in this mixed setting, potentially posing greater challenges for the sliding window detectors. Future work should investigate the performance of each sliding window approach under the real-life scenario to further investigate the performance of the sliding window detectors.

Finally, we acknowledge that in typical real-life scenarios of plagiarism detection, generated AI text will often be ‘modified’ by a human author in an attempt to obscure the AI origin of the text. Future research should attempt to simulate these human alterations by leveraging an English lexical database (such as WordNet [16]) to systematically modify AI text (e.g. by randomly deleting words or sentences and replacing words with their synonyms). Such an experiment would help determine whether AI text can still be detected even after human-like modifications.

¹<https://www.deepseek.com>

Bibliography

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.
- [2] Mohammed Al-Sarem, Faisal Saeed, Abdullah Alsaeedi, Wadii Boulila, and Tawfik Al-Hadhrami. Ensemble methods for instance-based arabic language authorship attribution. *IEEE Access*, 8:17331–17345, 2020.
- [3] Shlomo Argamon and Shlomo Levitan. Measuring the usefulness of function words for authorship attribution. In *Proceedings of the 2005 ACH/ALLC Conference*, pages 4–7, 2005.
- [4] Christopher M Bishop and Nasser M Nasrabadi. *Pattern recognition and machine learning*, volume 4. Springer, 2006.
- [5] Leo Breiman. Random forests. *Machine learning*, 45:5–32, 2001.
- [6] Lars Buitinck, Gilles Louppe, Mathieu Blondel, Fabian Pedregosa, Andreas Mueller, Olivier Grisel, Vlad Niculae, Peter Prettenhofer, Alexandre Gramfort, Jaques Grobler, Robert Layton, Jake VanderPlas, Arnaud Joly, Brian Holt, and Gaël Varoquaux. API design for machine learning software: experiences from the scikit-learn project. In *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, pages 108–122, 2013.
- [7] Qian Chen, Wen Wang, Mengzhe Chen, and Qinglin Zhang. Discriminative self-training for punctuation prediction. pages 771–775, 08 2021.
- [8] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, page 785–794, New York, NY, USA, 2016. Association for Computing Machinery.
- [9] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine learning*, 20:273–297, 1995.
- [10] Thomas Cover and Peter Hart. Nearest neighbor pattern classification. *IEEE transactions on information theory*, 13(1):21–27, 1967.
- [11] Chris Deotte. Accelerating k-nearest neighbors 600x using rapids cuml. <https://medium.com/rapids-ai/accelerating-k-nearest-neighbors->

- [600x-using-rapids-cuml-82725d56401e](#), February 2020. Accessed: 2025-02-06.
- [12] Heather Desaire, Aleesa E Chua, Madeline Isom, Romana Jarosova, and David Hua. Distinguishing academic science writing from humans or chatgpt with over 99% accuracy using off-the-shelf machine learning tools. *Cell Reports Physical Science*, 4(6), 2023.
- [13] Heather Desaire, Aleesa E Chua, Min-Gyu Kim, and David Hua. Accurately detecting ai text when chatgpt is told to write like a chemist. *Cell Reports Physical Science*, 4(11), 2023.
- [14] Joachim Diederich, Jörg Kindermann, Edda Leopold, and Gerhard Paass. Authorship attribution with support vector machines. *Applied intelligence*, 19:109–123, 2003.
- [15] Edinburgh Compute and Data Facility. Edinburgh compute and data facility web site. <https://www.ecdf.ed.ac.uk>, January 2025. Accessed: 2025-01-19.
- [16] C. Fellbaum. *WordNet: An Electronic Lexical Database*. Language, Speech and Communication. Mit Press, 1998.
- [17] Evelyn Fix. *Discriminatory analysis: nonparametric discrimination, consistency properties*, volume 1. USAF school of Aviation Medicine, 1985.
- [18] Sebastian Gehrmann, Hendrik Strobelt, and Alexander M Rush. Gltr: Statistical detection and visualization of generated text. *arXiv preprint arXiv:1906.04043*, 2019.
- [19] Shayan Gerami. Ai vs. human text. <https://www.kaggle.com/datasets/shanegerami/ai-vs-human-text>, 2024. Accessed: 2024-11-04.
- [20] Raji Ghawi and Jürgen Pfeffer. Efficient hyperparameter tuning with grid search for text categorization using knn approach with bm25 similarity. *Open Computer Science*, 9:160–180, 2019.
- [21] Jean Kaddour, Joshua Harris, Maximilian Mozes, Herbie Bradley, Roberta Raileanu, and Robert McHardy. Challenges and applications of large language models. *arXiv preprint arXiv:2307.10169*, 2023.
- [22] Wincenty Lutoslawski. Principes de stylométrie appliqués à la chronologie des œuvres de platon. *Revue des études grecques*, 11(41):61–81, 1898.
- [23] Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D Manning, and Chelsea Finn. Detectgpt: Zero-shot machine-generated text detection using probability curvature. In *International Conference on Machine Learning*, pages 24950–24962. PMLR, 2023.
- [24] Frederick Mosteller and David L Wallace. Inference in an authorship problem: A comparative study of discrimination methods applied to the authorship of the disputed federalist papers. *Journal of the American Statistical Association*, 58(302):275–309, 1963.

- [25] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [26] Sebastian Raschka, Joshua Patterson, and Corey Nolet. Machine learning in python: Main developments and technology trends in data science, machine learning, and artificial intelligence. *arXiv preprint arXiv:2002.04803*, 2020.
- [27] Anderson Rocha, Walter J. Scheirer, Christopher W. Forstall, Thiago Cavalcante, Antonio Theophilo, Bingyu Shen, Ariadne R. B. Carvalho, and Efstathios Stamatatos. Authorship attribution for social media forensics. *IEEE Transactions on Information Forensics and Security*, 12(1):5–33, 2017.
- [28] Gordon Ross. Lecture 1 - stylometry: An overview, Sep 2024.
- [29] Gordon Ross. Lecture 2 - introduction to classification, Sep 2024.
- [30] Santiago Segarra, Mark Eisen, and Alejandro Ribeiro. Authorship attribution through function word adjacency networks. *IEEE Transactions on Signal Processing*, 63(20):5464–5478, 2015.
- [31] Alex J Smola and Bernhard Schölkopf. A tutorial on support vector regression. *Statistics and computing*, 14:199–222, 2004.
- [32] Jasper Snoek, Hugo Larochelle, and Ryan P Adams. Practical bayesian optimization of machine learning algorithms. *Advances in neural information processing systems*, 25, 2012.
- [33] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [34] William H. Walters. The effectiveness of software designed to detect ai-generated writing: A comparison of 16 ai text detectors. *Open Information Science*, 7(1):20220158, 2023.
- [35] Shichao Zhang, Xuelong Li, Ming Zong, Xiaofeng Zhu, and Debo Cheng. Learning k for knn classification. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 8(3):1–19, 2017.
- [36] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.
- [37] Ying Zhao and Justin Zobel. Effective and scalable authorship attribution using function words. In *Asia Information Retrieval Symposium*, pages 174–189. Springer, 2005.

Appendices

Appendix A

A.1 Examples of Pipeline in Python Syntax

A.1.1 Support Vector Classifier (SVC)

```
1 svc_pipeline = Pipeline([
2     ('count_vectorizer', CountVectorizer(vocabulary =
3         functionWords100eng)),
4     ('normalizer', Normalizer(norm='l1')),
5     ('svc', SVC())
6 ])
7 svc_pipeline.fit(X_train, y_train)
```

Listing A.1: Python syntax of the pipeline and training for default SVC classification model.

A.1.2 K-Nearest Neighbours (KNN)

```
1 knn_pipeline = Pipeline([
2     ('count_vectorizer', CountVectorizer(vocabulary =
3         functionWords100eng)),
4     ('normalizer', Normalizer(norm='l1')),
5     ('knn', KNeighborsClassifier())
6 ])
7 knn_pipeline.fit(X_train, y_train)
```

Listing A.2: Python syntax of the pipeline and training for default KNN classification model.

A.1.3 Random Forest (RF)

```
1 rf_pipeline = Pipeline([
2     ('count_vectorizer', CountVectorizer(vocabulary =
3         functionWords100eng)),
4     ('normalizer', Normalizer(norm='l1')),
5     ('rf', RandomForestClassifier())
6 ])
7 rf_pipeline.fit(X_train, y_train)
```

Listing A.3: Python syntax of the pipeline and training for default RF classification model.

A.1.4 Extreme Gradient Boosting (XGBoost)

```
1 xgboost_pipeline = Pipeline([
2     ('count_vectorizer', CountVectorizer(vocabulary =
3         functionWords100eng)),
4     ('normalizer', Normalizer(norm='l1')),
5     ('xgboost', XGBClassifier())
6 ])
7 xgboost_pipeline.fit(X_train, y_train)
```

Listing A.4: Python syntax of the pipeline and training for default XGBoost classification model.

Appendix B

B.1 Hyperparameters Found From Tuning

B.1.1 Support Vector Classifier (SVC)

```
C=193.6973643718782  
kernel='rbf'  
gamma='scale'
```

B.1.2 K-Nearest Neighbours (KNN)

```
n_neighbors=1  
weights='uniform'  
p=1  
metric='manhattan'
```

B.1.3 Random Forest (RF)

```
n_estimators=127  
max_depth=30  
min_samples_split=2  
min_samples_leaf=1  
max_features='log2'
```

B.1.4 Extreme Gradient Boosting (XGBoost)

```
n_estimators=492  
max_depth=10  
learning_rate=0.14191431734842344  
subsample=0.9220460865286193  
colsample_bytree=0.5494819660911112  
min_child_weight=2.8745172998057233  
gamma=0.09436616074117617  
reg_alpha=0.2122087861565824  
reg_lambda=0.6878620633790371
```

Appendix C

C.1 Top 100 Features for XGBoost

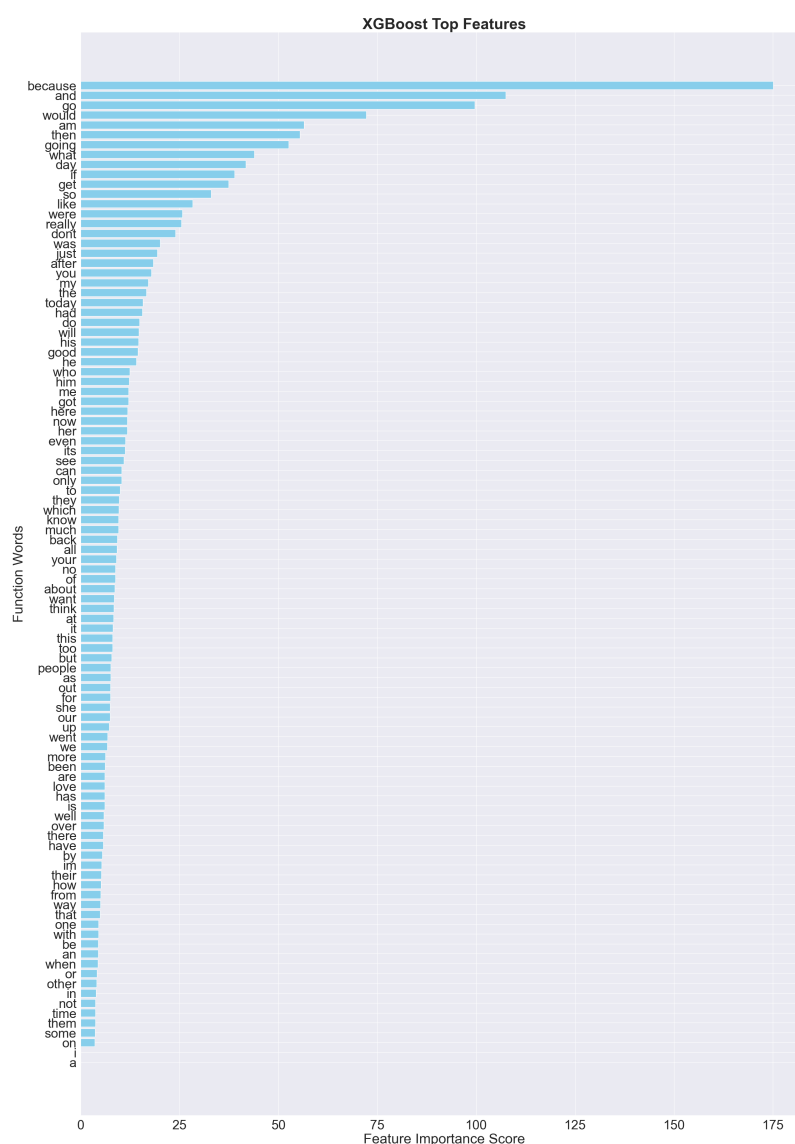


Figure C.1: The feature scores for all 100 function words when using XGBoost on the test set.